



Probing

Agent-Native 分布式AI训练诊断系统

让 Coding Agent 长出看穿分布式训练的眼睛和手
揭开 AI 性能的真相

版本

v0.2.5
(Beta)

代码规模

Rust 86K + Python 15K +
Web 21K

开源协议

Apache
2.0

日期

2026.07

汇报内容

从价值到技术到增长 — 用云原生语言理解 Probing

01 价值和意义

万卡训练时代的诊断范式革命。用 K8s 类比定位：Probing 之于 AI 训练诊断 $\approx$ K8s 之于容器编排 — 从碎片化工具到统一平台，但难度更高（不能停机）

02 技术方案和独特性

四层架构设计、没有北向流量的范式选择、五大创新设计原则、SQL 联邦查询、分布式火焰图、Python 变量跟踪、NCCL 等待分解 — 6 项业界首创

03 技术深度与竞品比较

10 款业界工具逐维度对比，总分 3.85 架构领先但成熟度差距明显。诚实评估五大短板，明确互补而非替代的定位

04 增长计划

目标用户：AI Infra 团队。框架无关统一接入，AI Agent 自动诊断。增长飞轮：部署 $\rightarrow$ 零改动接入 $\rightarrow$ 自动诊断 $\rightarrow$ Skill 沉淀。从工具到平台

万卡训练时代，传统诊断范式正在崩溃

GPU 规模从 256 扩展到 10,000+, 传统 "先收集再分析" 范式在三个维度同时失效

$O(N^2)$

数据传输爆炸

万卡全量 trace 达 TB 级，收集本身成为瓶颈

3h+

人工不可行

10K rank 逐个检查，即使每 rank 1 秒也需近 3 小时

误判

级联故障误归因

ring/tree 拓扑中 1 个慢点多跳传播，统计方法将受害者误判为罪魁祸首

核心矛盾：训练规模在指数级增长，但诊断工具仍停留在"生成火焰图 → 人工解读"的旧范式。当 GPU 数量超过 1,000 时，没有任何人类专家能有效遍历所有 rank 的性能数据。

传统 Profiler 的困境

PyTorch Profiler、Nsight Systems 生成 GB 级 trace 文件，需要人工通过 GUI 分析；不支持结构化查询；不支持跨 rank 联邦聚合；无 AI Agent 接口；诊断知识不可复用。

需要的范式转换

从 "收集 → 分析" 转向 "查询 → 归因 → 闭环"：查询推送到数据所在位置，利用执行模型做确定性归因，AI Agent 自主完成诊断闭环。

Probing : Agent-Native 诊断范式

不是又一个 Profiler，而是让 AI Agent 成为诊断流程一等公民的基础设施



三大用户价值

- 研究员** 调试训练不稳定性，实时检查张量值和梯度，无需停止训练
- 框架开发者** 零侵入分析框架内部，在生产环境中调试用户报告的问题
- MLOps** 无需重启服务即可监控，跨节点关联性能问题

安全边界设计

MCP 接口默认只读：仅接受 SELECT/WITH/SHOW/DESCRIBE，结果有大小上限。

干预类工具（set_config、eval_python）需显式设置 `PROBING_MCP_ALLOW_WRITE=1` 才启用，且每次写入都有审计日志。

Agent 能放手诊断，只有在你明确授权时才可改动训练任务。

如果用云原生语言理解 Probing

Probing 之于 AI 训练诊断 $\approx$ K8s 之于容器编排 — 从碎片化工具到统一平台

K8s / 云原生 $\rightarrow$ Probing

Kubelet in every Node $\rightarrow$ Probe in every Rank

kubectl (声明式 YAML) $\rightarrow$ SQL (声明式查询)

CRD / Operator $\rightarrow$ Skills (steps.yaml)

CSI / CNI 标准接口 $\rightarrow$ ProbeDataSource 契约

KubeFed / Cluster API $\rightarrow$ global.* 联邦查询

Prometheus pull-based $\rightarrow$ 没有北向流量

Sidecar 注入 (webhook) $\rightarrow$ PROBING=1 / inject

OperatorHub / Helm Hub $\rightarrow$ Skills 社区市场

同一个级别的意义

K8s 统一了碎片化的容器编排生态: Mesos / Swarm / Nomad $\rightarrow$ 统一 API + 声明式模型。

Probing 正在统一碎片化的训练诊断生态: Nsight / PyTorch Profiler / Megatron profiler $\rightarrow$ 统一 SQL + Agent 接口。

两者都是从"每个团队写脚本"到"声明一个意图"的范式跃迁。

但难度更高

K8s 管理的是无状态容器——Pod 挂了可以重启，调度失败可以重试。

Probing 管理的是有状态训练进程——不能重启、不能中断、不能有可感知开销。万卡训练一次中断损失数十万元，zero-overhead 不是优化目标而是生存约束。

相当于在不能停机的生产数据库内部建一套 K8s 控制面。

一句话给老板：K8s 让部署从写脚本变成写 YAML；Probing 让诊断从写脚本变成写 SQL。K8s 管无状态容器可重启，Probing 管有状态训练进程不能中断——这是更硬的约束，也是更深的护城河。

项目规模与技术资产

Probing 已具备完整的系统级实现，覆盖从底层存储到 Agent 接口的全链路

86K

Rust 代码行
(不含 Web)

15K

Python SDK
代码行

21K

Web UI
(Dioxus WASM)

15

Workspace
Crates

诊断能力覆盖

预置诊断 Skill

13 个

MCP 工具

8 个

SQL 数据表

20+ 表

联邦标签列

6 个

开销不变量

6 条形式化

硬件与生态支持

NVIDIA NCCL

v3 + v4 ABI

华为昇腾 HCCL

Shim 兼容层

PyTorch

2.8+ 推荐

Agent 集成

Cursor/Claude/Codex

平台

Linux (macOS 部分)

安装即用： `pip install probing` — 零代码修改， `PROBING=1 python train.py` 即可启用， 或 `probing -t PID inject` 注入运行中进程。

四层分层架构 — 教科书级 Clean Architecture

依赖方向严格单向向下，6 个公共契约定义模块交互边界，组合根只存在于 engine.rs

L4

Experience 体验层 — skills/ (13个诊断剧本) · web/ (Dioxus WASM UI) · Python hooks ≈ Helm / Operator 生态
诊断 UX、声明式知识库、可视化

↓ 依赖方向

L3

Control 控制层 — probing/server (HTTP/MCP) · probing/cli (命令行) ≈ API Server + kubectl
HTTP API、MCP 工具、注入、联邦 fan-out

↓

L2

Collectors 采集层 — probing-python · probing-nccl-profiler · probing-gpu · probing-cc ≈ CSI / CNI 驱动
指标采集：Torch trace、NCCL proxy、GPU 利用率、系统指标

↓

L1

Platform 平台层 — probing-core (SQL引擎/联邦) · memtable (无锁存储) · proto (wire types) · macros ≈ etcd + controller manager
Apache DataFusion SQL 引擎、联邦三路径路由、MEMT 二进制格式

6 个公共契约

关键设计原则

@table Python 数据表

skills YAML 诊断剧本

proto DTOs 线协议

Federation tags 联邦标签

核心技术能力

从系统层做起：免插桩注入、SQL 化遥测、NCCL 等待分解、跨节点联邦 — 数据天生是给 Agent 消费的结构



动态探针注入

运行时附加到进程，无需代码修改。通过 Python .pth hook 或 probing inject 注入。支持远程进程。



SQL 查询引擎

嵌入 Apache DataFusion，支持 JOIN/窗口函数/CTE/子查询。自定义表只需实现 TableProvider trait。



联邦三路径路由

AST 级 SQL 分析：聚合下推 $O(1)$ 、流式扫描 $O(N)$ 、广播 $O(N^2)$ 。自动注入联邦标签列和查询护栏。



NCCL 等待分解

导出 ncclProfiler_v3/v4 C 符号，零侵入采集。Proxy Step Wait 三段分解：SendGpuWait → PeerWait → SendWait。



Shadow Step 校准

4:1 节奏 A/B 对照：4 个 probed step + 1 个 shadow baseline，自动量化探测开销本身。



MCP Agent 接口

8 个 MCP 工具：query/run_skill/list_tables 等。写入操作受环境变量控制，默认只读。



声明式诊断剧本

13 个预置 Skill：health_overview, slow_rank, nccl_culprit_victim 等。YAML + MD 双文件，可在 CLI/Web/MCP 三端运行。



无锁 mmap 存储

自描述 MEMT 格式：环形缓冲 + generation 计数。单写者 + 无锁读者，零拷贝零序列化跨进程共享。



延迟 GPU 事件读取

采样步延迟 3~16 步后异步读取 GPU 事件，将 sync 频率从"每步一次"降低到"每 16 步一次"。

杀手级功能 — 三个鲜活场景

不是抽象的"低开销采集", 而是老板能看懂的具体能力: 分布式火焰图、Python 变量跟踪、分布式 Query 热力图

🔥 分布式火焰图



跨 rank 栈自动聚合: 128 个 rank 执行相同代码的栈合并 count, 不同路径分叉显示。Python/Native 栈在 `_PyEval_EvalFrameDefault` 边界拼接。

```
-- 联邦查询所有 rank 的模块栈
SELECT module, stage,
       median(duration)
FROM global.python.torch_trace
WHERE local_step = 7
GROUP BY module, stage
```

🐍 Python 变量跟踪

```
# 运行时注入, 零代码修改
trace("train_loop",
      watch=["loss", "grad_norm"])
# loss 变化自动记录 ↓
step=42 loss=2.31
step=43 loss=2.18
step=44 loss=1.95
```

运行时追踪任意变量: 变量 `id()` 变化即记录函数名、行号、值、类型。 `torch.Tensor` inplace 操作自动拦截。

```
-- 查询 loss 变更历史
SELECT lineno, variable_name,
       value, value_type
FROM python.trace_variables
WHERE variable_name = 'loss'
ORDER BY lineno
```

🌐 分布式 Query 热力图



一条 SQL 跨 rank 聚合: rank × step 矩阵, 颜色深浅 = 步时长。一眼识别 straggler 和异常 step。联邦下推 O(1) 传输。

```
-- 跨 rank 步时长热力图
SELECT _rank, global_step,
       avg(duration_ms)
FROM global.python.comm_collective
GROUP BY _rank, global_step
```

这三个功能的共同点: 数据天然结构化、可 SQL 查询的、跨 rank 联邦的。传统 Profiler 生成 trace 文件后需要人工用 GUI 分析; Probing 让 Agent 用一条 SQL 就能拿到跨 128 个 rank 的聚合结果 — 这就是 Agent-Native 的具体含义。

6 项业界首创 — 套壳工具伪造不出来的护城河

护城河在底下的系统层：活体注入、关系型遥测、NCCL 分解、跨节点联邦 — 这些是 Agent 集成能触达的基础

1

联邦 SQL 查询

业界首个支持跨 rank SQL 联邦查询的训练诊断系统。AST 级路由，聚合下推 O(1) 传输。

2

MCP 协议集成

业界首个为 AI Agent 设计的训练诊断接口。8 个 MCP 工具，Cursor/Claude Code/Codex 原生支持。

3

声明式诊断剧本

业界首个将诊断知识固化为可执行 YAML 的系统。13 个预置 Skill，可复用、可扩展。

4

Culprit/Victim 自动归因

业界首个从 NCCL 时间戳自动推导故障 rank 的系统。区分"谁慢"和"谁因别人慢而等待"。

5

Shadow Step 开销校准

业界首个自动化探测开销的训练采样器。4:1 A/B 对照，6 条形式化开销不变量 + 自动化测试守护。

6

跨硬件生态兼容

同时支持 NVIDIA NCCL (v3+v4 ABI) 和华为昇腾 HCCL。dlopen 拦截 libprofapi.so，映射到统一采集路径。

创新设计 (一): 没有北向流量 — 范式的根本选择

不是实现细节，而是架构哲学：数据不动，查询动

传统方案：北向汇聚

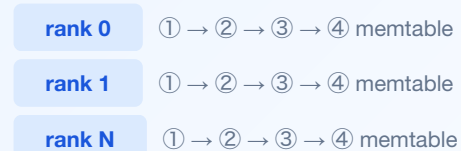


四大问题：

- 带宽 $O(N)$ 放大，中心节点网络瓶颈
- 中心节点单点故障 = 诊断全断
- 序列化+网络IO 侵入训练热路径
- 中心节点需存储全量数据

VS

Probing：数据留存在本地



训练期间零网络流量 ↓

诊断时 coordinator 按需拉取 (南向)

四大收益：

- 训练热路径只做 mmap append，开销可控
- memtable 内存有界，格式统一
- 进程退出后文件仍可 attach 查询
- 路径 A 聚合下推：peer 只返回少量行

有意的 Tradeoff

放弃实时 dashboard 的「好看」，换取训练热路径的「不干扰」。放弃跨 global 表 JOIN 的灵活性，换取联邦查询的正确性和可推理性。**这些约束是设计选择，不是技术缺陷。**

为什么是创新的

NVIDIA Nsight、PyTorch Profiler、Megatron 内置 profiling 全部走北向汇聚。Probing 是**第一个**把完整诊断栈嵌入训练进程、数据零上报、查询按需拉取的系统。这个范式选择决定了后续所有设计的走向。

创新设计 (二): 五大设计原则及其创新逻辑

每个设计都有明确的 tradeoff, 而且这些 tradeoff 是有意为之的

1 SQL 作为统一诊断接口

原则: 诊断逻辑写在 skill SQL 中, 不在 engine 硬编码分支。

创新理由: 传统 profiler 的诊断逻辑硬编码在工具里。Probing 让"找最慢模块"变成一条 SQL, 任何诊断逻辑都能被 **Agent** 自动化执行。engine.rs 只负责注册和路由, 新诊断 = 新 skill SQL, 不需要改引擎代码。

```
SELECT module, median(duration)
FROM python.torch_trace
WHERE stage LIKE 'post forward%'
GROUP BY module ORDER BY 2 DESC
```

3 六列联邦标签自动注入

原则: 合并结果时自动附加 6 列标签, 但显式列清单不自动带。

创新理由: 这是一个**有意的设计约束** — 防止用户在不知情情况下拿到带标签的列导致聚合出错。SELECT * 自动展开标签方便探索; 明确指定列时则严格只返回请求的列。安全与灵活的精确平衡。

_host

_addr

_rank

_node_rank

_local_rank

_role

2 probe.* 与 global.* 的二象性

原则: 同一组物理表的两个视图, 差异仅在于是否跨 rank fan-out。

创新理由: coordinator 收到 global.* 时改写为 probe.* 发给 peer。**peer 侧永远执行 probe.*** — 从根本上杜绝递归联邦。不是两张表, 而是同一张表的安全视图切换。

probe.*
本地扫描
单 rank 调试

global.*
联邦 fan-out
跨 rank 聚合

4 联邦三路径 — SQL AST 级选型

原则: 不是字符串匹配, 是解析 SQL 抽象语法树判断路由路径。

创新理由: 路径 A 检测到可 merge 的 GROUP BY → 各 peer 先聚合, 只返回少量行 (**80% 集群查询走这条路径**)。解析失败时**保守走 C** — 宁可慢, 不能错。这是分布式查询优化器在训练诊断场景的首次应用。

路径 A
聚合下推 O(1)

路径 B
联邦扫描 O(N)

路径 C
广播 O(N²)

5 Agent-Native 不是事后添加，是架构第一天就有的

原则：MCP 暴露的 query / run_skill / cluster_query 和 CLI / Web / skill 走的是同一个 **Engine::async_query** 入口。

创新理由：Agent 不需要学新 API，只需要会写 SQL 或调用 skill。默认只读，写入需 `PROBING_MCP_ALLOW_WRITE=1`。这不是给传统工具"加个 API"，而是从架构第一天就把 **Agent** 作为一等公民设计。

```
{
  "mcpServers": {
    "probing": {
      "url": "http://127.0.0.1:8080/mcp"
    }
  }
}
// 一行配置，Agent 即获得完整的诊断能力
```


技术深度 (一): 无锁存储 + 联邦路由

两个设计最精巧的组件，构成了低开销采集和跨节点查询的基础

MEMT 无锁存储格式

环形缓冲区 + generation 计数

固定 chunk 组成环形，回收时 generation+1，读者比较前后值检测并丢弃回收 chunk

单写者 + 无锁读者

写者 &mut self 排他，读者通过 Acquire/Release 内存序完全无锁

三种后端统一抽象

Heap (进程内) / Shm (shm_open 跨进程) / File (mmap 持久化)

per-chunk 字符串去重

DedupState 哈希去重，存储为 4 字节回引用，可节省 20%+ 空间

训练进程写入 mmap 文件，probing 服务器通过同一文件读取 — 零拷贝、零序列化

联邦三路径路由 (~3,066 行)

路径	场景	传输
A 下推	单表 global.* + SUM/COUNT/MIN/MAX	O(1)
B 扫描	单表 global.* 非聚合	O(N)
C 广播	JOIN/CTE/UNION/子查询	O(N²)

基于 `sqlparser-rs` AST 级分析（非字符串匹配）

查询护栏：强制 broadcast 需 LIMIT，自动注入 LIMIT 10000

每行联邦数据自动附加 6 个标签列：_host, _addr, _rank, _node_rank, _local_rank, _role

技术深度 (二): NCCL 等待分解 + Culprit/Victim 归因

整个项目中技术最硬核的部分：从低级时间戳数据到可操作诊断结论的跃迁

Proxy Step Wait 三段分解

■ **SendGpuWait** — proxy 等待本地 GPU 完成 send buffer
Culprit 信号：本地 GPU 慢

■ **SendPeerWait** (v4) — 等待对端 rank 的 network buffer
接收方拥塞信号

■ **SendWait** — 实际发送操作耗时

首次进入优先设计：只有第一次进入某状态时才记录起始时间戳，正确处理 proxy progress loop 重试行为。

引用计数完成模型：通过 live_children + stopped 标志判断 collective 是否真正完成，处理异步完成语义。

Culprit/Victim 自动归因

Culprit (罪魁祸首)

send 边 (rank→peer) 上的高 send_gpu_wait_ns
本地 GPU 慢，导致其他 rank 等待

Victim (受害者)

recv 边上的高 recv_wait_ns
等待对端发送，自身无过错

Propagated Victim

recv wait 高但 peer 的 send_gpu_wait 也高
根因在上游

Receiver Congestion (v4)

高 send_peer_wait_ns
send 被 peer 的 clear-to-send 信用阻塞

Slot Pool 内存管理：预分配固定大小 slot pool，避免高频采集路径分配内存。三重验证 (bounds + liveness + 指针身份) 防止 stale handle。pool_pressure.rs 在 pool 接近满时自动降级采样率。分片锁 + try_lock 避免阻塞 NCCL 回调。

加权总分：Probing 3.85 架构领先，成熟度差距明显

10 维度加权评分，覆盖架构、采集、分析、分布式、AI Agent、开销、扩展、易用、生态、创新



诚实评估：架构设计 (4.5) 和分布式诊断 (4.5) 确实领先，但易用性 (3.0) 和生态成熟度 (2.5) 是明显短板。总分 3.85 领先但差距不大 — DeepFlow (3.38) 和 Nsight Systems (3.27) 在成熟度和用户基础上远超我们。**架构优势尚未转化为采用优势。**

逐维度评分：架构与分布式领先，易用与生态是短板

不回避弱项 — 诚实的自评比满屏 5.0 更有说服力



差异化能力（竞品尚未覆盖）

主要短板与互补关系

主要短板：

- 跨 rank 实时聚合需按需联邦查询（非持续推送）
- 0.x Beta, 缺乏大规模生产验证
- SQL 门槛高（skill 部分缓解）
- 无 Prometheus/Grafana 集成

差异 联邦 SQL 查询 — 跨 rank 声明式查询

差异 MCP 协议集成 — AI Agent 原生接口

差异 声明式诊断剧本 — 知识可复用

差异 Culprit/Victim 自动归因

差异 Shadow Step 开销校准

互补而非替代：

Probing 作为诊断入口和 Agent 接口；Nsight Systems 做深度下钻；NCCL Flight Recorder 补充层数据；Nsight Compute 做 kernel 级优化。

范式对比: Agent-Native vs 传统 Profiler

Probing 不是在做"又一个 profiler", 而是在重新定义诊断的交互范式

Probing (Agent-Native)

- 运行时实时查询, 在线诊断
- SQL + MCP + Skills 声明式接口
- AI Agent 自动执行诊断剧本
- 联邦查询跨 rank 聚合
- Skills 可扩展诊断知识库
- Shadow Step 自动校准开销
- Culprit/Victim 确定性归因
- 支持 NVIDIA + 华为昇腾

VS

传统 Profiler

- 生成 trace 文件, 事后离线分析
- 图形界面 (Nsight GUI, TensorBoard)
- 人工解读火焰图/时间线
- 单进程视角, 无联邦聚合
- 固定分析维度, 不可扩展
- 开销固定, 无法校准
- 依赖统计相关性推断
- 通常仅支持 NVIDIA

定位： Probing 不是替代 Nsight Systems 或 PyTorch Profiler，而是补全它们缺失的维度 — 分布式联邦视角、Agent 原生接口、声明式诊断知识复用。深度下钻仍需 Nsight，kernel 级优化仍需 Nsight Compute。Probing 的价值在于**把诊断从"人看图"变成"Agent 查 SQL"**，以及跨 rank 的联邦聚合能力。

增长策略：谁在用 Probing，为什么用

不是给单个研究员用的 profiler，是给 AI Infra 团队用的统一诊断平台

目标用户：AI Infra / 平台团队

谁：百卡以上 GPU 集群的 AI 平台 / Infra 团队

场景：支撑多个业务团队，每个团队用不同框架训练

痛点：

- 每个框架自带 profiler，接口不统一
- Megatron 的问题 Nsight 看不懂
- 业务方报"训练慢"，Infra 团队无从下手
- 人工诊断 3h+，排障靠经验传承

Probing 的价值主张

一个工具，统一接入所有框架：

- `PROBING=1` 一行环境变量，不改业务代码
- PyTorch / Megatron / DeepSpeed / FSDP 通吃
- 所有框架的数据统一成 SQL 表

AI Agent 自动诊断：

- 业务方报"慢" → Agent 自动跑 skill → 给结论
- Skill = 可复用的诊断知识，经验沉淀
- Infra 团队不用每个 case 都人肉看图

增长飞轮

Infra 团队部署
一次部署，全集群可用



业务方零改动接入
PROBING=1 即可



Agent 自动诊断
Skill 沉淀经验



更多 Skill → 更多场景
正反馈飞轮

系统性 Infra 解决方案：框架无关的统一诊断层

不管业务方用什么框架，Probing 都能统一接入、监控、诊断和优化

框架无关 — 统一接入

探针注入训练进程，hook PyTorch runtime 层，**不依赖具体框架实现**。无论业务方用 Megatron、DeepSpeed、FSDP 还是 raw PyTorch，采集到的数据 schema 完全一致。

Megatron-LM
PP/TP/DP

DeepSpeed
ZeRO 1/2/3

FSDP
原生分布式

Raw PyTorch
DDP / custom

任何框架，同一个入口

```
PROBING=1 python train.py # Megatron
PROBING=1 deepspeed train.py # DeepSpeed
PROBING=1 torchrun train.py # FSDP
```

AI 驱动 — 自动诊断与优化

Infra 团队不需要人肉看图。业务方报问题 → Agent 连 MCP → 自动跑诊断 skill → 给出结论和建议。

Step 1 业务方: "训练慢了"
→ Agent 连接 Probing MCP

Step 2 Agent 自动跑 health_overview
→ job_health → slow_rank --global

Step 3 定位到 rank 42 是 straggler
→ nccl_culprit_victim 自动归因

Step 4 结论: "rank 42 NCCL recv_wait 占 73%,
建议检查 IB 连接"
→ 全程 < 30 秒, 无人介入

Web UI = 更可靠的实时看板

每个 rank 内嵌 HTTP server, Web 直连查看实时状态。**比中心化 dashboard 更可靠**: 无单点故障、无聚合管道断裂、无中心存储瓶颈。集群视角通过联邦查询按需聚合, 不持续占用网络。

从工具到平台 — 和 K8s 一样的路径

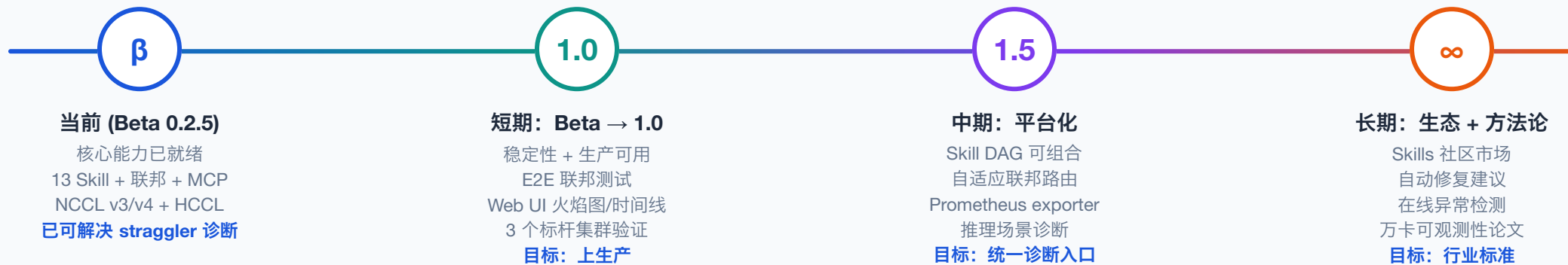
K8s 编排工具 → 平台 → 行业标准 (CRD + OperatorHub)

Probing 诊断工具 → 诊断平台 → 训练可观测性标准 (Skill + MCP)

短期: 解决 straggler 急性痛点 → 中期: Infra 统一诊断入口 → 长期: Skill 社区市场 + 自动修复。**每个新 Skill = 一个新 Operator, 让平台更聪明。**

产品路线图：从诊断工具到训练可观测性平台

每阶段都有明确的价值交付，不是为技术而技术



关键里程碑

- M1** 3 个 100+ 卡集群标杆验证 (Megatron / DeepSpeed / FSDP 各一)
- M2** Skill 数量 13 → 30, 覆盖 Top 10 训练故障场景
- M3** Agent 自动诊断闭环: 报障 → 诊断 → 结论 < 1 分钟
- M4** 推理场景 P99 延迟诊断能力

学术价值

五原则万卡可观测性方法论框架（查询驱动可观测性、执行模型感知归因、校准式开销预算、声明式诊断知识、Agent 原生接口），当前实现完整度约 51%。**Gap 是论文贡献而非缺陷** — 每个原则的未完成部分都是独立的研究方向。

学术价值：从"工具实现"到"万卡可观测性方法论"

五原则方法论框架，当前实现完整度约 51% — gap 是论文贡献而非缺陷

P1

查询驱动
可观测性

联邦下推
只传结果不传数据

40%

P2

执行模型
感知归因

NCCL proxy 语义
确定性因果归因

50%

P3

校准式
开销预算

Shadow Step
闭环反馈控制

45%

P4

声明式
诊断知识

YAML 剧本
可组合可复用

50%

P5

Agent
原生接口

MCP + 结构化查询
写操作安全控制

70%

论文叙事策略

Gap 是贡献，不是缺陷。 论文核心贡献不是"我们做了一个工具"，而是"我们识别了万卡训练可观测性的范式转换，提出了五原则方法论，并通过 Probing 在生产规模上验证了其可行性"。

方法论文论结构：Introduction → Motivation → Methodology (核心 3 页) →

最优先实现的两个 Gap

优先级 1：拓扑感知归因算法 (P2)

从 peer + channel_id 构建通信拓扑图，沿拓扑边做因果传播分析。万卡级联故障的核心解法，直接与简单排序做 ablation 对比。

优先级 2：自适应联邦路由 (P1)



总结：Probing 的核心价值

价值与意义

Probing 之于 AI 训练诊断 $\approx$ K8s 之于容器编排。从碎片化工具（Nsight/PyTorch Profiler/Megatron）到统一 SQL + Agent 接口。但约束更硬：K8s 管无状态容器可重启，Probing 管有状态训练进程不能中断、不能有可感知开销——这是更深的护城河。

技术独特性

四层架构 + 6 契约 + 没有北向流量 + SQL 统一接口 + `probe.*/global.*` 二象性 + 联邦三路径 AST 路由 + 无锁 mmap 存储 + 分布式火焰图 + Python 变量跟踪 + 分布式热力图 + NCCL 等待分解 + 声明式 Skill + MCP 接口。6 项业界首创构成护城河。

技术领先性

10 款工具逐维度对比，加权总分 3.85 领先但差距不大。架构与分布式诊断是核心优势，易用性 (3.0) 和生态成熟度 (2.5) 是最大短板。差异化能力 5 项，但尚未转化为采用优势。

增长前景

目标用户：AI Infra / 平台团队（百卡以上集群）。框架无关统一接入（PROBING=1 通吃 Megatron/DeepSpeed/FSDP），AI Agent 自动诊断闭环（<1 分钟出结论）。增长飞轮：Infra 部署 $\rightarrow$ 业务方零改动接入 $\rightarrow$ Agent 自动诊断 $\rightarrow$ Skill 沉淀经验 $\rightarrow$ 更多场景。路径和 K8s 一致：诊断工具 $\rightarrow$ 诊断平台 $\rightarrow$ 训练可观测性标准（Skill 生态 + MCP 接口）。